

Guía Proyecto Final Parcial I: Desarrollo de API REST Empresarial

Integración de Spring Boot, Clean Architecture y DevSecOps bajo el Modelo de Calidad de McCall (Ciclo PDCA)

Docente: Ing. Diego L. Gamboa

dlgamboa1@espe.edu.ec

Departamento Ciencias de la Computacion - ESPE

Índice

1. Introducción y Enfoque Diferencial	2
1.1. El Modelo de McCall como Eje Transversal	2
2. PLAN (PDCA) + CALIDAD (McCall)	2
2.1. Definición del Problema y Alcance	2
2.2. Mapeo de Requerimientos a Factores de McCall	3
3. CONFIGURACIÓN DEL ENTORNO	3
3.1. Dependencias Core (pom.xml)	3
4. DISEÑO DE ARQUITECTURA (Clean Architecture)	3
5. DO (IMPLEMENTACIÓN)	4
5.1. 1. Capa de Dominio (Domain)	4
5.2. 2. Capa de Aplicación (Application)	4
5.3. 3. Capa de Interfaces (Controllers)	5
6. PRUEBAS AUTOMATIZADAS (Testability & Reliability)	5
7. CHECK (PDCA) - MÉTRICAS Y CALIDAD	6
8. DEVSECOPS PIPELINE	6
9. QUALITY SCORING ENGINE	6
10. ACT (PDCA) - MEJORA CONTINUA	7
11. CAPTURAS SIMULADAS Y VALIDACIÓN	7
11.1. Checklist de Validación Final	8

1. Introducción y Enfoque Diferencial

El presente documento constituye una guía técnica rigurosa para la construcción de una API REST empresarial(educativa) para la **Gestión de Tareas (Task Management)**, **Inventario**, **Financieros**, **E-Commerce**, etc. Se aborda el ciclo de vida completo del software a través del marco **PDCA (Plan-Do-Check-Act)**, asegurando que cada decisión arquitectónica y línea de código sea trazable a los factores de calidad del **Modelo de McCall**.

1.1. El Modelo de McCall como Eje Transversal

El Modelo de McCall categoriza la calidad del software en tres perspectivas:

1. **Operación del Producto:** Correctitud (Correctness), Confiabilidad (Reliability), Eficiencia (Efficiency), Integridad (Integrity), Usabilidad (Usability).
2. **Revisión del Producto:** Mantenibilidad (Maintainability), Flexibilidad (Flexibility), Testabilidad (Testability).
3. **Transición del Producto:** Portabilidad (Portability), Reusabilidad (Reusability), Interoperabilidad (Interoperability).

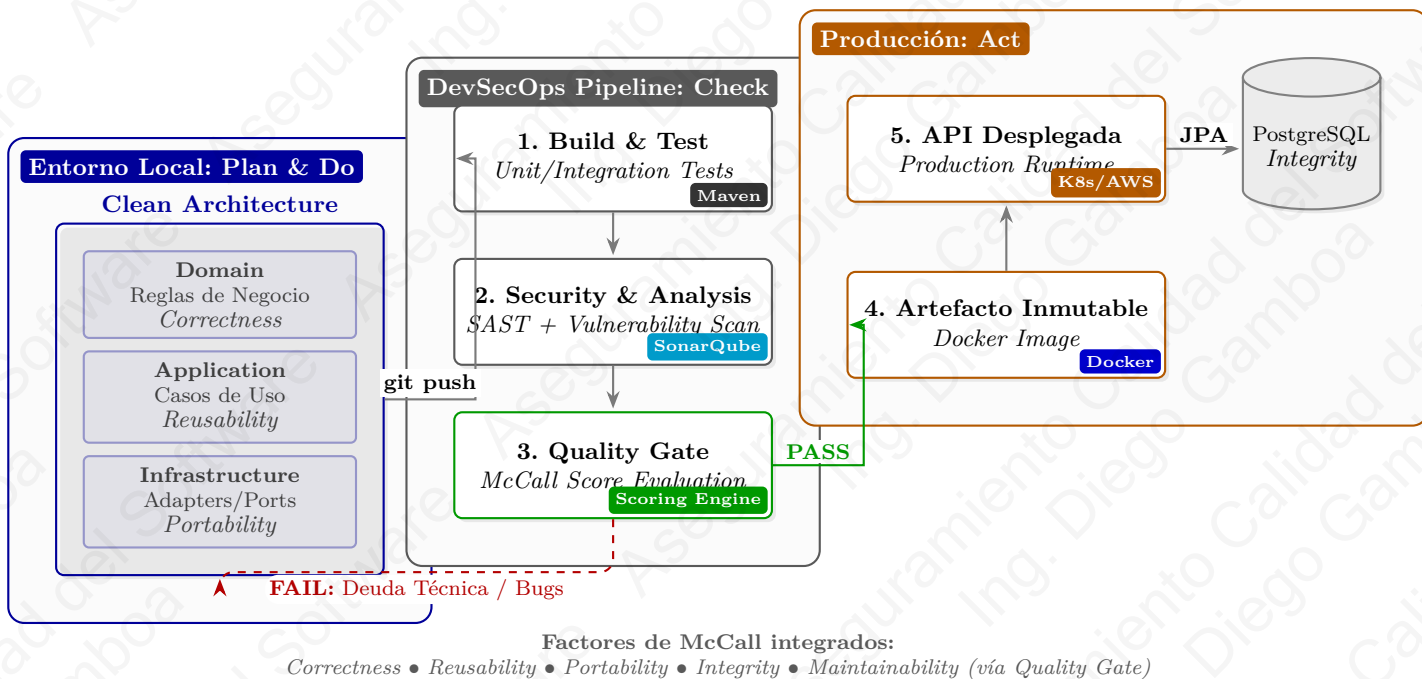


Figura 1: Arquitectura de flujo CI/CD con validación del Modelo de McCall. El pipeline DevSecOps evalúa métricas de calidad antes del despliegue; los fallos retroalimentan al equipo para mejora continua (PDCA).

2. PLAN (PDCA) + CALIDAD (McCall)

2.1. Definición del Problema y Alcance

Las organizaciones requieren gestionar las tareas/procesos de forma concurrente, segura y auditable. El proyecto consistirá en un CRUD de Tareas/Procesos con asignación de estados y validación de

seguridad (JWT).

2.2. Mapeo de Requerimientos a Factores de McCall

Requisito	Factor McCall	Métrica	Validación
Cifrado y Auth JWT	Integrity	0 Vulnerabilidades Críticas	SAST (SonarQube/SpotBugs)
Respuesta < 200ms	Efficiency	Tiempo de latencia (<i>p95</i>)	Pruebas de Carga (JMeter)
Cálculo exacto de fechas	Correctness	0 Defectos Funcionales	Unit Tests (JUnit)
Arquitectura Hexagonal	Maintainability	Deuda Técnica < 5 %	SonarQube Code Smells
Despliegue Dockerizado	Portability	Tiempo de despliegue	CI/CD Pipeline Success
Alta cobertura de código	Testability	Line Coverage > 85 %	JaCoCo Report

Tabla 1: Trazabilidad Requisitos vs Calidad

3. CONFIGURACIÓN DEL ENTORNO

Para garantizar la **Portabilidad** y **Reusabilidad**, estandarizar el entorno:

- **JDK:** Java 25 (LTS) - Beneficios de Virtual Threads para *Efficiency*.
- **Framework:** Spring Boot 4.0.x.
- **Build Tool:** Maven.

3.1. Dependencias Core (pom.xml)

```
1 <dependencies>
2   <!-- Web y Core -->
3   <dependency>
4     <groupId>org.springframework.boot</groupId>
5     <artifactId>spring-boot-starter-web</artifactId>
6   </dependency>
7   <!-- Persistencia (Integrity, Reliability) -->
8   <dependency>
9     <groupId>org.springframework.boot</groupId>
10    <artifactId>spring-boot-starter-data-jpa</artifactId>
11  </dependency>
12  <!-- Validaciones (Correctness) -->
13  <dependency>
14    <groupId>org.springframework.boot</groupId>
15    <artifactId>spring-boot-starter-validation</artifactId>
16  </dependency>
17  <!-- Herramientas (Maintainability) -->
18  <dependency>
19    <groupId>org.projectlombok</groupId>
20    <artifactId>lombok</artifactId>
21    <optional>true</optional>
22  </dependency>
23 </dependencies>
```

Listing 1: Fragmento de pom.xml

4. DISEÑO DE ARQUITECTURA (Clean Architecture)

Aplicamos Arquitectura Limpia para potenciar la **Mantenibilidad**, **Testabilidad** y **Flexibilidad** (Modelo de McCall). La regla de dependencia exige que las capas externas dependan de las internas, nunca al revés.

Estructura de Directorios

```
com.empresa.tasks
|-- domain          (Reglas de negocio puras, Entidades - Correctness)
|-- application     (Casos de uso, Puertos/Interfaces - Reusability)
|-- infrastructure  (JPA, DB, APIs externas - Portability)
|-- interfaces      (Controladores REST, DTOs - Interoperability)
```

5. DO (IMPLEMENTACIÓN)

5.1. 1. Capa de Dominio (Domain)

Impacto McCall: **Correctness** (las reglas de negocio no se corrompen).

```
1 package com.empresa.tasks.domain.model;
2
3 import lombok.AllArgsConstructor;
4 import lombok.Data;
5 import lombok.NoArgsConstructor;
6 import java.time.LocalDateTime;
7
8 @Data
9 @AllArgsConstructor
10 @NoArgsConstructor
11 public class Task {
12     private Long id;
13     private String title;
14     private String description;
15     private TaskStatus status;
16     private LocalDateTime createdAt;
17
18     public void markAsCompleted() {
19         if (this.status == TaskStatus.CANCELLED) {
20             throw new IllegalStateException("Cannot complete cancelled task");
21         }
22         this.status = TaskStatus.COMPLETED;
23     }
24 }
```

Listing 2: Task.java

5.2. 2. Capa de Aplicación (Application)

Impacto McCall: **Reusability y Flexibilidad**.

```
1 package com.empresa.tasks.application.usecases;
2
3 import com.empresa.tasks.domain.model.Task;
4 import com.empresa.tasks.application.ports.out.TaskRepositoryPort;
5 import lombok.RequiredArgsConstructor;
6 import org.springframework.stereotype.Service;
7 import org.springframework.transaction.annotation.Transactional;
8
9 @Service
10 @RequiredArgsConstructor
11 public class TaskUseCase {
12
13     private final TaskRepositoryPort taskRepository;
14
15     @Transactional
16     public Task createTask(Task task) {
17         // Business logic validation (Correctness)
18         task.setStatus(TaskStatus.PENDING);
19         return taskRepository.save(task);
20     }
21 }
```

Listing 3: TaskUseCase.java

5.3. 3. Capa de Interfaces (Controllers)

Impacto McCall: **Interoperability** (APIs REST estándar) e **Integrity** (Validación de entrada).

```
1 package com.empresa.tasks.interfaces.rest;
2
3 import jakarta.validation.Valid;
4 import lombok.RequiredArgsConstructor;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.web.bind.annotation.*;
7
8 @RestController
9 @RequestMapping("/api/v1/tasks")
10 @RequiredArgsConstructor
11 public class TaskController {
12
13     private final TaskUseCase taskUseCase;
14
15     @PostMapping
16     @ResponseStatus(HttpStatus.CREATED)
17     public TaskResponseDTO create(@Valid @RequestBody TaskCreatedDTO dto) {
18         Task task = TaskMapper.toDomain(dto);
19         Task createdTask = taskUseCase.createTask(task);
20         return TaskMapper.toDto(createdTask);
21     }
22 }
```

Listing 4: TaskController.java

6. PRUEBAS AUTOMATIZADAS (Testability & Reliability)

Las pruebas garantizan la **Confiabilidad** del software ante cambios.

```
1 package com.empresa.tasks.application.usecases;
2
3 import com.empresa.tasks.domain.model.Task;
4 import com.empresa.tasks.application.ports.out.TaskRepositoryPort;
5 import org.junit.jupiter.api.Test;
6 import org.junit.jupiter.api.extension.ExtendWith;
7 import org.mockito.InjectMocks;
8 import org.mockito.Mock;
9 import org.mockito.junit.jupiter.MockitoExtension;
10 import static org.mockito.ArgumentMatchers.any;
11 import static org.mockito.Mockito.*;
12 import static org.junit.jupiter.api.Assertions.*;
13
14 @ExtendWith(MockitoExtension.class)
15 class TaskUseCaseTest {
16
17     @Mock
18     private TaskRepositoryPort taskRepository;
19
20     @InjectMocks
21     private TaskUseCase taskUseCase;
22
23     @Test
24     void shouldCreateTaskSuccessfully_CorrectnessValidated() {
25         // Arrange
26         Task task = new Task(null, "Test", "Desc", null, null);
27         when(taskRepository.save(any(Task.class))).thenReturn(task);
28
29         // Act
30         Task result = taskUseCase.createTask(task);
31
32         // Assert (Correctness & Reliability)
33         assertNotNull(result);
34         verify(taskRepository, times(1)).save(task);
35     }
36 }
```

Listing 5: TaskUseCaseTest.java (Prueba Unitaria)

7. CHECK (PDCA) - MÉTRICAS Y CALIDAD

Para el componente "Check", integramos análisis estático.

Herramienta	Métrica	Factor McCall	Umbral Aceptado
JaCoCo	Code Coverage	Testability	$\geq 85\%$ en Domain/App
SonarQube	Code Smells	Maintainability	Grado A (< 5 % Deuda)
SpotBugs	Bugs / Defectos	Correctness	0 Blocker/Critical
OWASP Dep. Check	Vulnerabilidades	Integrity	0 CVEs detectados

Tabla 2: Métricas de Calidad y Herramientas

8. DEVSECOPS PIPELINE

Pipeline en **GitHub Actions** para garantizar Integridad y Portabilidad mediante CI/CD continuo.

```
1 name: Java DevSecOps Pipeline
2 on: [push, pull_request]
3 jobs:
4   build-and-test:
5     runs-on: ubuntu-latest
6     steps:
7       - uses: actions/checkout@v3
8       - name: Set up JDK 21
9         uses: actions/setup-java@v3
10        with:
11          java-version: '21'
12          distribution: 'temurin'
13       - name: Build and Test with Maven
14         run: mvn clean verify
15       - name: SonarQube Analysis
16         env:
17           SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
18         run: mvn sonar:sonar
```

Listing 6: Fragmento de ci-cd.yml

9. QUALITY SCORING ENGINE

Implementación de un motor Java para calcular matemáticamente la calidad basada en McCall.

```
1 package com.empresa.tasks.sqa;
2
3 public class QualityScoringEngine {
4
5     // Pesos asignados segun importancia del proyecto
6     private static final double WEIGHT_CORRECTNESS = 0.3;
7     private static final double WEIGHT_TESTABILITY = 0.3;
8     private static final double WEIGHT_MAINTAINABILITY = 0.2;
9     private static final double WEIGHT_INTEGRITY = 0.2;
10
11     public static double calculateScore(double coveragePercent, int bugs, int smells, int
12     vulns) {
13         double testabilityScore = Math.min(coveragePercent, 100.0);
14         double correctnessScore = Math.max(100.0 - (bugs * 10), 0);
15         double maintainabilityScore = Math.max(100.0 - (smells * 2), 0);
16         double integrityScore = vulns == 0 ? 100.0 : 0.0; // Tolerancia 0
17
18         return (testabilityScore * WEIGHT_TESTABILITY) +
19             (correctnessScore * WEIGHT_CORRECTNESS) +
20             (maintainabilityScore * WEIGHT_MAINTAINABILITY) +
21             (integrityScore * WEIGHT_INTEGRITY);
22     }
23 }
```

10. ACT (PDCA) - MEJORA CONTINUA

Las acciones resultantes (Act) tras analizar el Quality Score; aplicar el **Ciclo PDCA para automatizar el flujo del proceso** de refactorización:

- **Mantenibilidad (Maintainability):** Si los *Code Smells* aumentan, programar un Sprint técnico de *Refactorización*.
- **Integridad (Integrity):** Si OWASP reporta fallos, actualizar bibliotecas (bump version) en el `pom.xml`.
- **Eficiencia (Efficiency):** Implementar *Caching* (Redis) si los tiempos de respuesta de la API superan los 200ms.

11. CAPTURAS SIMULADAS Y VALIDACIÓN

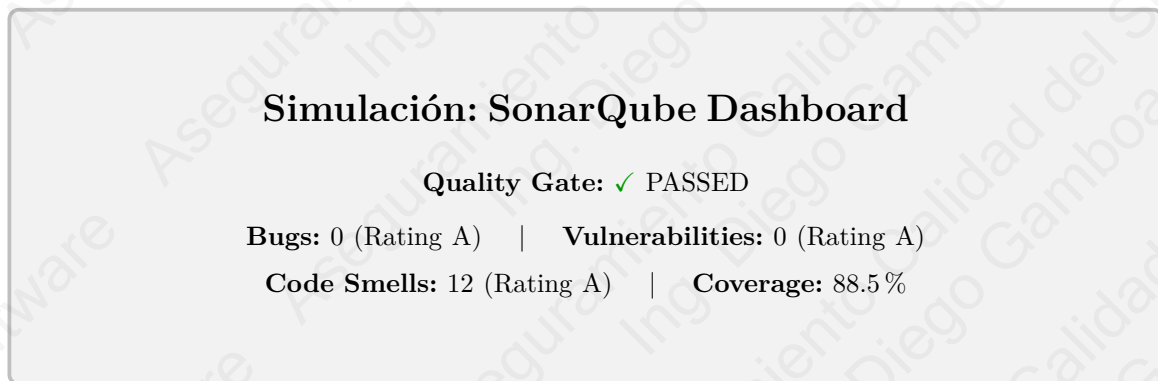


Figura 2: Validación de Factores de Revisión e Integridad.

11.1. Checklist de Validación Final

✓ Checklist de Entrega

- ☒ **Proyecto compila** (Maven Build Success).
- ☒ **API responde** (HTTP 201 Created en POST).
- ☒ **Tests pasan** (100 % JUnit pass rate).
- ☒ **SonarQube OK** (Quality Gate Passed, sin severidades altas).
- ☒ **Pipeline CI/CD exitoso** (Artifact Dockerizado).
- ☒ **Integración de todas las herramientas y correcto flujo de trabajo** (Automatización de los procesos).
- ☒ **Documento PDF** (Informe Final):
 - **Introducción:** Contexto del problema, objetivos y alcance del MVP.
 - **Metodología:** Descripción del ciclo PDCA + aplicación del Modelo de McCall.
 - **Resultados:**
 - Métricas de calidad obtenidas (cobertura, bugs, vulnerabilidades)
 - Quality Score calculado por el motor
 - Evidencias de ejecución (capturas de SonarQube, pipeline)
 - **Discusión:**
 - Análisis de factores McCall más críticos
 - Limitaciones encontradas y decisiones arquitectónicas
 - Comparativa con enfoques alternativos
 - **Conclusiones:**
 - Lecciones aprendidas sobre calidad medible
 - Recomendaciones para escalar el framework
 - Trabajo futuro (ej: integrar ISO 25010, IA en testing, etc)

Nota de Evaluación

No se dispone de una rúbrica específica de evaluación. La valoración se realizará considerando la ejecución integral del proceso, la coherencia metodológica, la trazabilidad de las actividades y la consistencia técnica de la solución desarrollada de inicio a fin. Incluyendo el Informe Final, que demuestra el sustento técnico del desarrollo del presente proyecto.